The typed term IR: code as the runtime representation

Status: P3d implemented (D167); P3c (D166); P3b (D165); P3a (D163); P2 (D162); P1 (D161); P4–P8 planned. This document defines the target representation, the kernel/Prelude split, the verification pipeline, and the migration stages. P1 adds the core term shape and its canonical portable document; P2 adds the kernel structural verifier, the `VerifiedTermProgram` receipt, the `scale_verify` entry, and the ABI; P3a adds the token-to-term boundary; P3b makes the active program and caller frames `Term list` and executes numbers and words. Text quotations and the temporary token projection remain until P3c/P3d; stack typing lands in P4.

The decision

The live program becomes a list of typed terms:

```
Term  = Literal of value * TermType
      | Word of signature
      | Quote of Term list
      | Call of Term list
      | Branch of Term * Term * Term
      | ...
Program = Term list
```

`apply` on a quotation splices its `Term list` onto `Program`. It is a typed list operation, not text parsing and not a second evaluator. A quotation is a value of the program representation, so code-as-data is the runtime representation, not a side channel.

The existing general IR already has the essential shape: `ExpressionAst` has `Quotation of body: Expression list` and `Block of body: Expression list` (`components/core/expression-ast.fs:23`). The defect is that the live `EvaluationState.Program` is `string list` (`components/core/runtime-types.fs:522`) and `quotation` stores text (`components/prelude/evaluator/control.fs:46`). The migration makes the term list authoritative; it does not add a fifth representation.

Verification

The term list is the unit of verification. P2 is the kernel structural verifier: it re-checks the canonical document, finite numbers, non-negative slots, captures and arities, non-empty word and closure names, and every quotation body recursively. It does not type a stack yet. P4 adds the signature environment and the stack-effect checker; a term list that passes P2 is well-formed, not yet stack-safe.

The kernel owns the trusted checker and issues a `VerifiedTermProgram` receipt. A term list without a receipt is refused. The Prelude may run a richer language-level checker (domain types, effects, staging), but it can only reject more: it cannot authorize a term list the kernel rejects. The kernel re-checks the core terms and the declared signatures; it does not trust a Prelude receipt.

A receipt is canonical JSON: `{"schema":1,"form":"term-receipt","digest":"<sha-256 of the canonical term-program>"}`. The digest binds the exact program bytes. Verification is idempotent: the same program yields the same digest. The `scale_verify` entry decodes a term document, verifies it, and writes the receipt; a malformed document returns `SCALE_INVALID` with a diagnostic. A splice does not exist before P3; because P2 verifies every quotation body recursively, P3's `apply` can consume only a verified body from a verified program.

The domain verifier (SMT, `FieldType`, refinement) remains a separate property. It reasons about values and contracts, not about the stack effect of the term list. A term list that type-checks is safe to execute; it is not thereby proved against its domain contract.

The kernel / Prelude split

Concern	Kernel	Prelude
Term encoding	The core `Term`/`TermList` image schema: tags, fields, quotation tree, bounds, and the portable term document.	The surface `ExpressionAst` and the elaboration from language terms to core terms. High-level constructs are lowered here.
Execution	The term-list loop, the program counter, the stack, quotation values, and splice. The kernel owns the machine safety of the loop.	Word semantics, user words, closures, control policy, actors, classes, dispatch, conditions, and the standard vocabulary.
Verification	The structural verifier and the stack-effect checker over core terms. It validates tags, references, quotation shape, stack bounds, and the declared signature table. It issues the receipt.	The language type rules: word signatures, argument and result types, effect tiers, dynamic-code policy, and the decision to request verification.
Dynamic code	A `verify` entry that checks a term program and returns a receipt; the executor refuses an unverified splice.	Parsing external text to terms, building runtime quotations, caching receipts, and attaching the `Verified`/`Unverified` effect.
Macros and staging	None. The kernel sees only expanded terms.	Compile-time application of a `Quote` to a term list, expansion, and the stage boundary. Expansion happens before final verification.
AOT	Executes only a verified term program. No JIT-only path is reachable from the AOT entry point.	The writer consumes `VerifiedTermProgram` and emits the AOT artifact. The term list is the source of truth; lowering is a compilation detail.
ABI	The term document, the verification receipt, and the verification entry are part of the ABI and the image schema. A change bumps the single version.	Uses the ABI. No host adapter contains language semantics.

The kernel must not gain the parser, the macro expander, the domain lattice, SMT, word implementations, classes, dispatch, conditions, actors, or a second evaluator. The Prelude must not own the kernel's structural verifier, receipt, stack safety, or ABI framing. The split is the same trust boundary as D95: the kernel is the abstract machine; the Prelude is the language.

Why the core term set, not the surface AST

The kernel's term set is deliberately small and stable: literals, word calls, quotations, splice, branches, slots, and closures. A new language feature is a Prelude word or a lowering into that set, not a new kernel case, unless it changes machine behavior. This keeps the kernel verifier small and

the AOT contract reviewable. The surface AST may be richer; it elaborates to the core terms before execution or compilation.

What is data and what is execution state

A `Quote` is a code value; when it is stored in the image it is a term document, not text. The active `Program` is control state: it is a typed list, but it is never serialized. A reader re-threads the program counter and the term list from the image's verified units. A quotation that crosses an actor or instance boundary is portable code and carries its receipt or is re-verified on arrival.

The split is not only a way to support several host languages. Its wider benefits—trusted-base size, independent evolution, fault attribution, capability security, portability and conformance, performance specialization, live deployment, test leverage, organizational scaling, formal reasoning, incremental migration, heterogeneous IPC, observability, language pluralism, and multi-tenancy—are collected in the glossary under Microkernel / Prelude split.

Placement audit

The recent actor IPC and term work is mostly on the correct side of the boundary:

1. Kernel: `TermIr` and `TermPortable` are the executable term shape and its canonical document. They are unused by execution until P2/P3, which is expected for P1. They contain no word resolution, signature, effect, or Prelude semantics.
2. Prelude/core: `RemoteHandleTag`, `RemoteHandles`, and the `send-to` remote branch are language capabilities and belong here. `PendingTransport` is the one placement smell: it is a host-effect queue, not scheduler state, but it currently lives in `SchedulerState` to keep the record churn small. It should move to a host-effects field or a separate host-state record before P3.
3. Host: `Wire`, `ActorHost`, and `LanguageTransport` are the transport and effect edge. They belong in `ipc`, not the kernel or the Prelude; they contain no language semantics. The Elixir adapter is a peer/harness, not a Scale component.
4. Documents: `ExpressionAst`/`ExpressionPortable` is the surface IR; `TermIr`/`TermPortable` is the executable IR. P4 must make the lowering total and must not add a third portable code form. The term vectors live with the kernel; the wire vectors live with the host transport.

Effects and tiers

The language exposes a verification effect on code values:

1. `Verified` code is statically verified and may enter the AOT path without a runtime check.
2. `Unverified` code may be constructed at runtime; it must pass the kernel's verifier once before its first splice. The receipt is cached.
3. External text is parsed to terms, then verified; it is never spliced as raw text.
4. Unrestricted `eval` is a separate verified-interpreter path. It is not part of the AOT contract; a JIT backend would need a verified or certifying compiler.

The effect is the type-level view of the kernel receipt. The minimal enforceable form is a capability (`VerifiedTermProgram`), not a full row-polymorphic effect system. A general effect system can come later without changing the term list.

Macros and staging

A macro is a quotation applied to a term list at compile time. It expands to a new term list, which is then verified and compiled. This is the MetaOCaml/LMS shape: code values at stage n, run at stage n+1. Expansion happens before the final verification pass, so the AOT entry point sees only expanded, verified terms. No runtime macro expansion is allowed on the AOT path.

P1 contract: the core term shape

P1 fixes the term shape and the portable document. It is implemented (D161). It does not add execution or stack typing; P4 adds the signature table and the stack-effect types.

The core variants are: `LiteralNumber`, `LiteralText`, `LiteralAtom`, `Word`, `Quote`, `Branch`, `SlotGet`, `SlotSet`, `MakeClosure`, `CallClosure`, and `Fail`. A quotation is `Quote of Term list`. `apply` is a word that splices a quotation in P3.

The portable form is canonical JSON:

```
{"schema":1,"form":"term-program","terms":[
  {"term":"number","value":2},
  {"term":"word","name":"add"}
]}
```

A `Quote` nests a `terms` list; `Branch` has `condition`, `then`, and `else`; `MakeClosure` has `name`, `captures`, and `arity`; `Fail` has `reason`. The canonical order is the order written by the encoder; the reader rejects a missing field, a wrong type, an unknown term, a negative slot, a negative capture, a bad schema, and a bad form.

The vectors under `components/kernel/term-vectors/` are `number-add`, `quote`, `branch`, `closure`, `fail`, and the negative `bad-schema` and `unknown-term`. P1 is shape and serialization only: no term is executed and no stack effect is checked.

P2 contract: the kernel verifier and receipt

P2 is implemented (D162). `TermVerification.verify` takes a `TermProgram` and returns a `VerifiedTermProgram` or a named refusal. The receipt is opaque: the record representation is private, so only `verify` can construct one. It carries the program and a lowercase SHA-256 digest of the canonical term-program bytes.

The structural checks are: the canonical document round-trips; a number is finite; a slot, capture, or arity is non-negative; a word or closure name is not blank; and every quotation body is checked recursively at its path. The verifier uses the same canonical bytes the portable reader accepts, so a term document cannot be verified in one spelling and executed in another.

`scale_verify` is exported at the current ABI version (2). It takes a term-program document and an output pair; on success it writes the `term-receipt` JSON and returns `SCALE_OK`. A malformed document, an oversized document, a null output pair, or an invalid handle returns `SCALE_INVALID`; a malformed document also writes a diagnostic text. The receipt binds the program digest, not a host object.

P2 deliberately does not claim stack safety. Stack underflow, quote input and output types, closure arity, and an `apply` effect need the P4 signature table and stack-effect types. There is no splice term before P3; P2 recursively verifies every quotation body, so a P3 splice cannot introduce an unverified body.

Migration

1. **P0 — record the model.** This document and D160. No code change.
2. **P1 — core term type and portable document.** Define the core `Term` and `TermProgram`; add a canonical portable form and golden vectors. No execution change. Implemented (D161).
3. **P2 — kernel structural verifier and receipt.** Re-check the canonical document, reject non-finite numbers, negative slots/captures/arities, empty word/closure names, and malformed quotation bodies. Issue `VerifiedTermProgram` with a canonical `term-receipt`. Add a `verify` ABI

entry and bump the version. Implemented (D162). Stack underflow, quote input types, closure calls, and `apply` remain P4 because they need the signature table and stack-effect types.

4. **P3 — runtime representation.** Change `EvaluationState.Program` to `Term list`; make `quotation`, `apply`, `define`, `control`, and `if` term operations. Remove the text-splice path. `apply` consumes only a verified quotation body. This phase is staged:
   1. **P3a — the term boundary.** Add the token-to-term boundary (`EvaluatorProgram`), make numeric literals `LiteralNumber` and names `Word`, and prove the projection used by the temporary kernel bridge. The active state still uses `Program: string list` until P3b. Implemented (D163).
   2. **P3b — typed control state.** Change `EvaluationState.Program` and the caller-program frames to `Term list`; the loop executes a number or a word and the existing control words splice terms through a temporary token projection. Quotation values are still text in P3b. Implemented (D165); the differential corpus keeps its text cases and converts at the interpreter boundary.
   3. **P3c — code values.** Store a quotation as a tagged term document, not token text; `quotation`, `if`, `apply`, `define`, and `control` consume code values. History and rollback store terms structurally. Implemented (D166).
   4. **P3d — remove the side channels.** Delete `toTokens`, the legacy text quotation reader, and the token-vector history encoding. Implemented (D167). Surface clause parsing and the kernel bridge keep their token projection.
5. **P4 — signature table and stack typing.** Add the language signature table and the stack-effect checker, including stack underflow, quote input and output types, and closure arity; lower `ExpressionAst` to core terms.
6. **P5 — verification effect.** Add `Verified`/`Unverified` code values, the runtime check-once path, and the external-text path.
7. **P6 — macros and staging.** Add compile-time quote application and expansion before final verification.
8. **P7 — AOT writer.** Make the writer consume `VerifiedTermProgram`; the interpreter and the compiled path share the term list.
9. **P8 — remove the side channels.** Delete token-string execution, text quotations, and `LoweredInstructions.lowerTokens` as runtime paths. The lowered IR becomes a compilation target of verified terms, not a source of semantics.

P3 contract: the runtime term list (staged)

P3a is implemented (D163). `EvaluatorProgram.ofTokens` maps a token list to core terms at the edge: a finite numeric token becomes `LiteralNumber`; every other token becomes `Word`. A non-finite number is a named refusal, never a silent NaN. `EvaluatorProgram.toTokens` is the temporary projection for the kernel bridge; it accepts only the two term shapes P3a produces and refuses any other term by name. The projection is a migration seam, not the target runtime.

P3a does not change `EvaluationState.Program`. P3b changes it to `Term list` and makes the loop execute a number or a word; P3c replaces text quotation values with tagged term documents; P3d removes `toTokens`, the token-vector history encoding, and `splitText` from execution. A term list is not executable until P3b, and a quotation value is not a term document until P3c.

P3b contract: the typed control state

P3b changes `EvaluationState.Program` and the caller-program fields on `HandlerFrame` and `MethodFrame` from `string list` to `Term list`. The loop executes exactly two term shapes:

1. `LiteralNumber value` pushes a scalar tensor; the token text is no longer reparsed.

2. `Word name` calls the existing word dispatcher.

Every other term is not executable in P3b. The only constructors that write `Program` are `EvaluatorProgram.ofTokens` and the control words, so a `Quote`/`Branch`/`SlotGet` in the active state is an invariant violation, not a language program. P3c adds `Quote` execution and P4 adds the typed stack rules.

The control words still use quotation text in P3b. `quotation`, `if`, `apply`, `define`, and `control` project their term lists to tokens with the temporary `EvaluatorProgram.toKernelTokens`, run the existing text logic, and convert the result back with `ofTokens`. The same projection feeds the kernel bridge and the token-vector history. `toKernelTokens` is total for `LiteralNumber` and `Word`; any other term is a programmer invariant failure, never a guessed token.

History remains a token-vector record in P3b: `record` takes the current term and the resuming term list, projects both, and stores the same seven-field step; rollback rehydrates the token vector with `ofTokens`. P3c replaces that encoding with a term document. Bookmark records `Word "bookmark"` and the typed program.

The P3b test batch covers a number/word program through `Evaluator.run`, the control words after conversion, bookmark rollback, and the kernel bridge. It also checks that a term that is not a number or word is refused at the projection seam rather than silently dropped. The full migration is mechanical but touches roughly every `Program` construction and every caller-program frame; P3d removes the temporary projection.

P3b non-goals: grouping `{ ... }` into `Quote`, executing a `Quote` or `Branch`, tagged code values, structural history, removing quotation text, stack typing, and the AOT path. Those are P3c, P3d, and P4.

P3c contract: quotation code values and structural history

P3c is implemented (D166). A quotation is a code value, not a text tensor. A code value is the tagged tensor `[QuoteTag; documentId]`; `documentId` is a text entry holding the canonical `term-program` document for the body. `QuoteTag` is `-34.0`. `EvaluatorCode.encode` turns a `Term list` into that value; `EvaluatorCode.decode` returns the terms and refuses anything that is not a well-formed code value. `Quote` is now executable: executing a `Quote` pushes its code value.

The control words consume code values directly:

1. `{ ... }` collects a term tree, including nested `Quote` terms, and pushes a code value.
2. `if` decodes the selected branch and splices its terms.
3. `apply` decodes a code value and splices its terms; a closure remains a separate case.
4. `define` stores the code value in the user-word table; expansion decodes it.
5. `control` and `lambda` store a closure whose quotation is a code value; `startCall` decodes it.
6. actor bodies, condition handlers, method bodies, predicates, guards, and set quotations decode their code values before use.

History becomes structural. The step's program field points at a code value for the resuming term list, and its current-term field points at a code value for the one-term program. Rollback decodes the code value; a legacy token-vector step is readable during P3c and removed in P3d.

The pattern and set clause parsers still use their surface token grammar. They decode the code value to terms and then use the temporary token projection for that grammar; no quotation is stored as text. P3d removes the projection from execution and the legacy history reader.

P3c non-goals: removing the kernel bridge projection, removing the surface clause parser, stack typing, the verification effect, macros, and AOT. Those are P3d, P4, and later.

P3d contract: remove the text side channels

P3d is implemented (D167). It removes the migration seams that P3c kept:

1. `EvaluatorCode.decode` no longer accepts a plain text tensor. A quotation is exactly `[QuoteTag; documentId]`; a text entry is a text value, not code.
2. `EvaluatorProgram.toTokens` is deleted. The active runtime and tests use `toKernelTokens` only where a surface token grammar or the kernel bridge still requires it.
3. `EvaluatorHistory` deletes the token writer and the legacy token-vector reader. A step's current-term and program fields are code values; a step that is not a code value is a named history refusal.
4. String quotation construction is removed from tests and helpers. A test that wants a quotation encodes a `Term list` with `EvaluatorCode.encode`.

The surface clause parsers for pattern matching and set literals may still project terms to tokens while they keep their existing grammar; that projection is a parser boundary, not a stored quotation. The kernel bridge keeps its token projection until the kernel term loop replaces it. P3d does not remove those two boundaries, and P4 adds stack typing.

P3d non-goals: removing the surface clause parser, the kernel term loop, stack typing, the verification effect, macros, and AOT.

P4 contract: signature table and stack typing

P4 adds the language signature table, the stack-effect checker, and the lowering from `ExpressionAst` to core terms. A term list that passes P2 is well-formed; P4 makes it stack-safe.

Invariants to preserve

1. The kernel refuses to execute a term list without a valid receipt. P2 issues the receipt; P3 enforces it in the runtime.
2. The kernel verifier is the execution authority; the Prelude cannot authorize a term list the kernel rejects.
3. A quotation is a term list; applying it is splicing, not parsing.
4. A term list that type-checks is safe to execute; domain verification is a separate property.
5. Self-modification invalidates the receipt; the changed list is re-verified before it runs.
6. A dynamic term cannot enter the AOT path without verification and a receipt.
7. The kernel gains no Prelude state, logic, parser, macro expander, or domain type system.

Risks

1. **A fifth representation.** The core term type must be the executable authority; `ExpressionAst`, `LoweredInstructions`, and the token list must become front-end or target views, not parallel runtimes.
2. **Term typing.** Stack effects and literal/quote types must be defined independently of the domain `FieldType` lattice; conflating them would make both weaker.
3. **Verification context.** A term list is not verifiable without a signature table and the image's word declarations. The table is part of the program document, not a hidden Prelude assumption.
4. **Self-modification.** A `Verified` value is a receipt, not a permanent property of a mutable list. Any replacement of a term invalidates it.
5. **AOT.** Dynamic terms cannot be AOT-compiled. They remain on the verified-interpreter path; a JIT is a separate, optional backend.
6. **ABI cost.** The term document, receipt, and verify entry are ABI and image-schema changes. They require one version bump and a conformance update.

Relationship to other documents

`docs/general-ir.typ` is the portable IR and the four-representation model. `docs/kernel/abi.typ` and `components/kernel/scale_abi.h` are the kernel contract. `docs/INVARIANTS.typ` is the constitution; the receipt rule is a proposed invariant until P2 lands. `docs/binary-format.typ` is the wire, which is orthogonal: the term list is code, the wire is transport. D160 records this decision.